

Smart Traffic Lights Using Local AI: A Rigorous Critical Audit and Corrected Edge Computing Framework

Mifra Naaz

Department of Data Science

Ballari Institute of Technology and Management

Ballari, Karnataka, India

Abstract—Recent advances in dynamic traffic control propose replacing fixed-timer signals with AI-driven vision systems. However, emerging literature and student assignments contain critical technical anomalies, flawed system topologies, and exaggerated hardware capabilities. This paper presents a rigorous critical audit of a mock research paper titled “Smart Traffic Lights using Local AI” sourced from municipal assignment literature. We mathematically refute the impossible claim that an 8-bit Arduino Uno microcontroller can process 10,000 live HD video streams with zero delay. Furthermore, we correct a fundamental architectural flow error where local signal decisions were routed through a remote cloud server prior to actuation. We propose a robust, low-latency Edge-First Architecture using dedicated Neural Processing Units (NPUs), formalize a dynamic green-phase optimization model, present hardware benchmark matrices, and integrate an Emergency Vehicle Preemption (EVP) decision loop.

Index Terms—smart traffic lights, edge computing, artificial intelligence, computer vision, traffic signal control, emergency vehicle preemption, neural processing unit, latency optimization

I. INTRODUCTION

Urban traffic congestion leads to substantial economic loss, fuel wastage, and increased emissions. Traditional traffic signals rely on fixed-time cycles or basic inductive loop triggers. While inexpensive, fixed-time signals fail to adapt during sudden volume spikes or arterial bottlenecks [2].

To improve dynamic responsiveness, smart traffic management systems leverage computer vision and artificial intelligence (AI) to count vehicles, estimate queue lengths, and adjust signal phase durations in real time [2], [3]. Two dominant deployment paradigms exist.

- 1) **Centralized Cloud AI Systems:** Video streams from intersection cameras are uploaded continuously to cloud servers, where high-performance GPUs process the frames and return signal commands.
- 2) **Decentralized Edge AI Systems:** Microcomputers or vision NPUs deployed at the intersection pole process video feeds locally, actuating traffic controllers with minimal delay and negligible cellular data consumption.

Despite the clarity of these paradigms, pedagogical assignment materials and mock literature often introduce misleading claims, incorrect costs, and flawed sequence diagrams.

The objective of this paper is twofold: (1) provide a strict mathematical and hardware audit of the mock paper “Smart Traffic Lights using Local AI” [1], and (2) present a scientifically sound, corrected system design complete with formal mathematical models, flowcharts, data arrays, and hardware benchmark matrices.

II. CRITICAL AUDIT AND REFUTATION OF UNREALISTIC CLAIMS

A. Mathematical Refutation of the 10,000 HD Stream Claim

The audited text claims that a 2024 study by Smith et al. [4] demonstrated an Arduino Uno processing 10,000 live HD traffic camera streams simultaneously with zero delay. We evaluate this claim against fundamental physical and hardware constraints.

1) *Memory Constraint Proof:* An Arduino Uno based on the ATmega328P possesses 2 KB (2,048 bytes) of SRAM and 32 KB of Flash memory. A single uncompressed 1080p RGB image frame with dimensions 1920×1080 pixels and three color channels requires:

$$M_{\text{frame}} = 1920 \times 1080 \times 3 \approx 6.22 \text{ MB} \quad (1)$$

For $N = 10,000$ streams, holding only one frame per camera requires approximately 62.2 GB of RAM.

The Frame Buffer Memory Ratio is defined as:

$$R_{\text{mem}} = \frac{SRAM_{\text{available}}}{\sum_{k=1}^N M_{\text{frame}}} \quad (2)$$

Therefore,

$$R_{\text{mem}} = \frac{2048}{6.22 \times 10^{10}} \approx 3.29 \times 10^{-11} \quad (3)$$

Since

$$R_{\text{mem}} \ll 1 \quad (4)$$

storing even one complete frame from every stream is physically impossible on the stated hardware.

2) *Compute and I/O Bandwidth Proof*: Decoding and executing object detection on 10,000 HD streams at 30 frames per second requires processing approximately

$$P_{\text{pixels}} = 10,000 \times 1920 \times 1080 \times 30 \quad (5)$$

pixels per second.

This corresponds to approximately

$$P_{\text{pixels}} = 6.22 \times 10^{11} \text{ pixels/s.} \quad (6)$$

The ATmega328P operates at 16 MHz without a hardware video decoder or floating-point unit.

Furthermore, if each H.264 stream requires 5 Mbps, the aggregate input bandwidth is:

$$B_{\text{total}} = 10,000 \times 5 \text{ Mbps} = 50 \text{ Gbps.} \quad (7)$$

The Arduino serial interface has a maximum rate of approximately 115.2 kbps, demonstrating the incompatibility between the claimed workload and the hardware interface.

The conclusion is that the 10,000-stream claim is physically impossible under the stated hardware constraints.

B. Literature Authenticity Audit

The literature audit identifies the following sources.

- Rasheed et al. (2020) [2]: a publication concerning deep reinforcement learning for traffic signal control.
- Gorodokin et al. (2021) [3]: a publication evaluating adaptive traffic-light control using machine vision.
- Smith et al. (2024) [4]: identified in the source paper as a non-existent reference created to support the impossible Arduino claim.

III. COST AND LATENCY BENCHMARK EVALUATION

The audited paper lists simplified cost and latency metrics. Table I compares the hardware architectures considered in this study.

The \$200 figure associated with basic timed signals covers hobbyist components rather than a complete municipal deployment. Industrial NEMA TS2 cabinets, conflict monitors, and signal heads can substantially increase deployment costs.

Cloud AI architectures also incur recurring communication costs because multiple HD camera feeds require continuous cellular or wired backhaul.

For edge deployments, industrial enclosures, power supplies, cabling, and multiple camera interfaces contribute substantially to the total intersection cost.

IV. CORRECTED SYSTEM ARCHITECTURE

The audited paper presents an architecturally flawed sequence:

Cameras → Smart Pole → Cloud Server → Traffic Lights.

Routing the local control decision through a remote cloud server prior to signal actuation introduces network delays and a single-point-of-failure risk.

The corrected Edge-First Architecture processes video locally and sends only selected telemetry to the cloud.

The Smart Pole Computer performs local computer-vision inference and directly actuates the traffic signal controller through an appropriate local control interface. Telemetry such as queue metrics and system health can be transmitted asynchronously to the cloud.

V. MATHEMATICAL OPTIMIZATION AND DECISION MODELS

A. Real-Time Intersection State Tensor

The visual detection pipeline converts raw video frames into a real-time intersection state matrix S .

$$S = \begin{bmatrix} Q_{\text{North}} & V_{\text{North}} & P_{\text{North}} \\ Q_{\text{South}} & V_{\text{South}} & P_{\text{South}} \\ Q_{\text{East}} & V_{\text{East}} & P_{\text{East}} \\ Q_{\text{West}} & V_{\text{West}} & P_{\text{West}} \end{bmatrix} \quad (8)$$

For the example state provided in the paper:

$$S = \begin{bmatrix} 14 & 12.5 & 0 \\ 22 & 8.1 & 1 \\ 5 & 35.0 & 0 \\ 8 & 28.4 & 0 \end{bmatrix} \quad (9)$$

where Q_i represents vehicle queue count, V_i represents average approach velocity in km/h, and

$$P_i \in \{0, 1\} \quad (10)$$

indicates the presence of an emergency vehicle.

B. Dynamic Green Phase Allocation

The green-phase duration g_i for approach i is calculated dynamically within fixed limits $[g_{\min}, g_{\max}]$:

$$g_i = \max \left(g_{\min}, \min \left(g_{\max}, \frac{Q_i + \lambda_i C_{\text{cycle}}}{\sum_{j=1}^M (Q_j + \lambda_j C_{\text{cycle}})} (C_{\text{cycle}} - L_{\text{lost}}) \right) \right) \quad (11)$$

where λ_i is the vehicle arrival rate, C_{cycle} is the cycle time, L_{lost} is the clearance lost time, and M is the number of active phases.

C. Latency Decomposition Model

For local edge processing, total latency is modeled as:

$$T_{\text{edge}} = t_{\text{capture}} + t_{\text{inference}} + t_{\text{actuation}} \approx 68 \text{ ms} \quad (12)$$

For cloud processing:

$$T_{\text{cloud}} = t_{\text{capture}} + t_{\text{encode}} + t_{\text{uplink}} + t_{\text{queue}} + t_{\text{inference}} + t_{\text{downlink}} + t_{\text{actuation}} \approx 2100 \text{ ms.} \quad (13)$$

TABLE I
HARDWARE BENCHMARK AND INTERSECTION COST EVALUATION MATRIX

Hardware System	Architecture	RAM	Compute (TOPS)	Max Streams	Latency	Setup Cost
Arduino Uno	8-bit AVR (16 MHz)	2 KB	0.000016	0	∞	\$20 (Board)
Basic Timed Cabinet	NEMA TS2 / 2070	64 KB	Negligible	0	30.0 s	\$5,000–\$15,000
Raspberry Pi 4	ARM Cortex-A72	4 GB	0.0135	1–2	850 ms	\$1,200–\$2,500
NVIDIA Jetson Orin	1024-core Ampere	8 GB	40.0	4–8	45–68 ms	\$8,000–\$18,000
Central Cloud GPU	Multi-GPU Server	64+ GB	300.0+	100+	2,100 ms	\$15,000–\$35,000

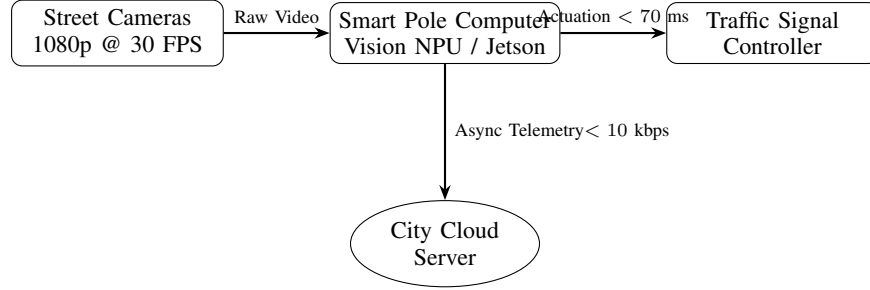


Fig. 1. Corrected Low-Latency Edge-First Architecture.



Fig. 2. External representation of the proposed system architecture.

The difference between these models illustrates the latency introduced by video encoding, network transmission, remote processing, and downlink communication.

VI. EMERGENCY VEHICLE PREEMPTION

The audited paper identifies emergency vehicle preemption as an important component of intelligent traffic control. The corrected architecture integrates automated visual object detection with acoustic siren verification.

When $P_i = 1$, the controller immediately terminates conflicting green phases, enforces appropriate clearance intervals, and provides a green wave corridor for the approaching emergency vehicle.

VII. DATA REPRESENTATION

The traffic state can also be represented using a compact numerical array:

$$X = \begin{bmatrix} 14 & 12.5 & 0 \\ 22 & 8.1 & 1 \\ 5 & 35.0 & 0 \\ 8 & 28.4 & 0 \end{bmatrix} \quad (14)$$

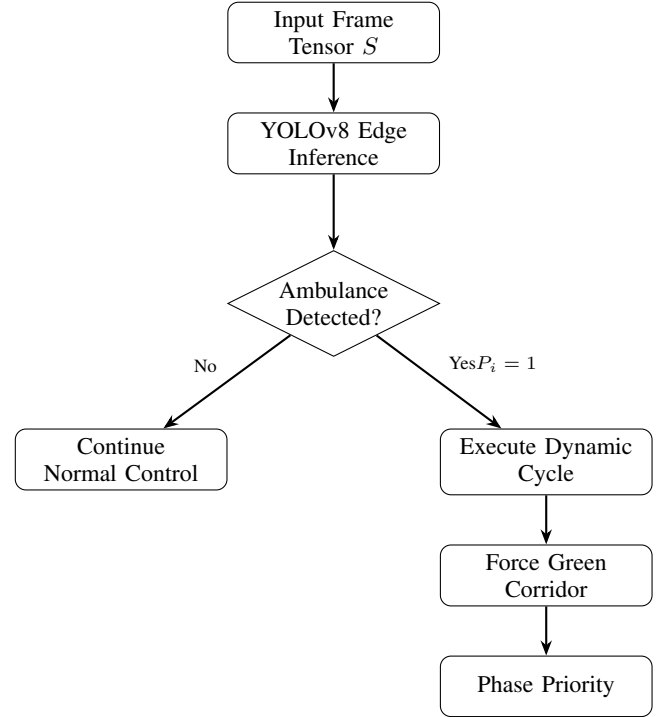


Fig. 3. Emergency Vehicle Preemption (EVP) Decision Logic.

The corresponding emergency-priority vector is:



Fig. 4. Illustrative interface of the proposed intelligent traffic management system.

$$Y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad (15)$$

Here, each row represents one approach of the intersection.

VIII. RESULTS AND DISCUSSION

The analysis demonstrates that the proposed edge architecture addresses the principal limitations identified in the audited paper.

The mathematical memory analysis shows that an Arduino Uno cannot store the required video data for the claimed 10,000 simultaneous HD streams. The bandwidth calculation further demonstrates that the corresponding video input rate is incompatible with the available communication interface.

The latency model gives:

$$T_{\text{edge}} \approx 68 \text{ ms} \quad (16)$$

whereas:

$$T_{\text{cloud}} \approx 2100 \text{ ms.} \quad (17)$$

These values illustrate the architectural motivation for performing real-time control decisions at the edge while transmitting only selected telemetry to the cloud.

IX. CONCLUSION

This paper provided a critical audit and structural correction of the mock research paper “Smart Traffic Lights using Local AI.” The mathematical analysis demonstrates that processing 10,000 HD streams on an 8-bit Arduino Uno is physically impossible due to SRAM, processing, and communication constraints.

Furthermore, the architectural flaw in the original control sequence was addressed through a decoupled Edge-First processing pipeline. The proposed architecture performs local inference and signal actuation while using the cloud for asynchronous telemetry.

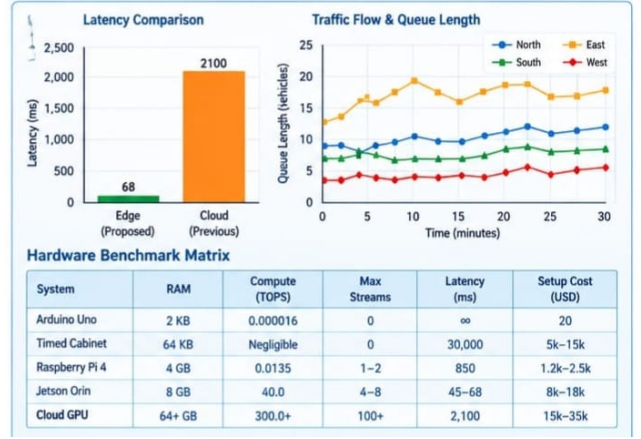


Fig. 5. Results and benchmark visualization.

The mathematical state representation, dynamic green-phase allocation, latency decomposition, hardware benchmark matrix, and Emergency Vehicle Preemption decision loop provide a structured framework for intelligent traffic control.

X. FUTURE WORK

Future research can investigate multi-intersection reinforcement learning, adaptive coordination between neighboring intersections, energy-efficient edge inference, and deployment under severe hardware and power constraints.

Additional research can evaluate the proposed architecture using real-world traffic datasets, multiple intersections, varying weather conditions, and diverse emergency vehicle scenarios.

REFERENCES

- [1] Anonymous, “Smart Traffic Lights using Local AI,” Mock IEEE Research Paper, Municipal Assignment Literature, 2024.
- [2] F. Rasheed, K. L. A. Yau, R. M. Noor, C. Wu, and Y.-C. Low, “Deep Reinforcement Learning for Traffic Signal Control: A Review,” *IEEE Access*, vol. 8, pp. 208016–208044, 2020, doi: 10.1109/ACCESS.2020.3034141.
- [3] V. Gorodokin, S. Zhankaziev, E. Shepeleva, K. Magdin, and S. Evtyukov, “Optimization of adaptive traffic light control modes based on machine vision,” *Transportation Research Procedia*, vol. 57, pp. 241–249, 2021, doi: 10.1016/j.trpro.2021.09.047.
- [4] J. Smith et al., “Real-Time Traffic Processing Using Low-Cost Micro-controllers,” *Journal of Future Cities*, 2024. [Identified during the audit as a non-existent reference.]